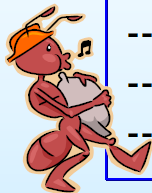


第6章：系统详细设计— 1.部件级设计

• 学习目的和要求



---部件级设计任务—**理解**

---基本设计原则—**掌握**

---设计模式—**掌握**

详细设计目标

对每个模块（用例）内部工作的描述成为详细设计。

- 确定应该怎样具体地实现所要求的系统。
- 详细设计阶段的任务不是具体地编写程序，而是要设计出程序的“蓝图”。
- 详细设计的结果基本上决定了最终的程序代码的质量。

一、部件级设计

1、部件级设计的任务

对软件体系结构中的部件（构件、连接件等）进行详细设计。

□ 部件在不同的分析设计方法中，对应着不同的名称

- 结构化分析和设计方法：部件被称为**模块**。
- 面向对象分析和设计方法：部件被称为**类或构件**。

□ 部件级设计的任务

- 软件构件是在体系结构设计过程中定义的。但没有在接近代码的抽象级上表示内部数据结构和每个构件的处理细节。
- 构件级设计定义了数据结构、算法、接口特征和分配给每个软件构件的通信机制。

一、部件级设计

1、部件级设计的任务

部件级设计中，设计者必须把需求模型和架构模型中的信息转换为一种设计表示，其步骤包括：

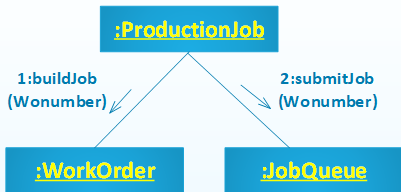
- **步骤1：**标识出所有与问题域相对应的设计类以及所有与基础设施域相对应的设计类。每个体系结构构件都需要细化。
- **步骤2：**细化所有不需要作为复用构件的设计类。详细描述实现细化需要的所有接口、属性和操作。
- **2.1：**在类和构件协作时说明消息的细节。某些情况下有必要通过对象间传递的消息结构说明协作细节。

一、部件级设计

1、部件级设计的任务

例：影印系统类协作消息细节

ProductionJob、WorkOrder和JobQueue三个对象相互协作，为生产线准备印刷作业。随着设计的进行，消息通过下列的扩展语法来细化：



[guard condition] sequence expression (return value) :=
message name(argument list)

守卫条件集

唤醒的操作

表明消息发送
序号的整数

传递给操作的
属性列表

由消息唤醒操作
返回的信息名

一、部件级设计

1、部件级设计的任务

- **2.2: 为每个构件确定适当的接口。**在构件设计中，一个UML接口是“一组外部可见（即公共的）操作”。接口不包括内部结构，没有属性。
- **2.3: 细化属性，并且定义实现属性所需要的数据类型和数据结构。**如果一个属性在多个设计类中重复出现，并且其自身具有比较复杂的结构，那么最好是为这个属性创建一个单独的类。
- **2.4: 详细描述每个操作中的处理流。**可能需要由程序设计语言的伪代码或UML的活动图来完成，每个构件都需要通过多次迭代进行细化。

一、部件级设计

1、部件级设计的任务

例：PrintJob打印纸张价格计算操作computeJobCost()

第一轮迭代，computeJobCost()可以采用下列方式扩展

`computeJobCost(weight,size,color):number`

如果实现computeJobCost()的算法简单且易于理解，则没有必要进行进一步的细化。如果算法比较复杂或难以理解，则需要结合活动图或伪代码进一步细化。

一、部件级设计

1、部件级设计的任务

例：PrintJob打印纸张

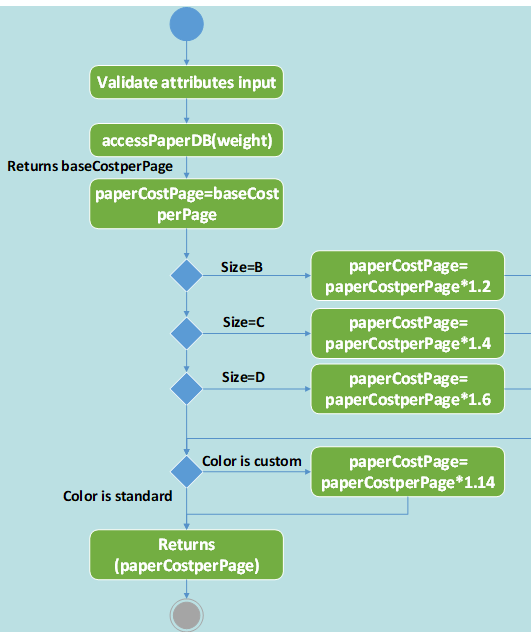
第一轮迭代，computeJobC

computeJobCost(weight,si

如果实现computeJobCost

要进行进一步的细化。

要结合活动图或伪代码进



一、部件级设计

1、部件级设计的任务

mybatis是数据映射器（数据映射器层：用于在对象和数据库之间搬运数据，同时保证对象、数据库和数据映射器层本身相对独立。） **mybatis**不是直接把类映射为数据库表，而是把**sql**语句的参数与结果（即输入与输出）映射为类。为如何在类和数据库间建立映射带来了更大的灵活性。同时也更好地隔离了数据库设计和应用程序中使用的对象模型。

需要对部署图进行细化，对执行的硬件和使用的操作系统环境加以说明，构件包再这个环境中的位置也需要确定。

□ **步骤5：**考虑每个构件级设计表示，并且时刻考虑其他可选方案。设计是一个迭代过程，一次迭代比上一次更精确。

一、部件级设计

2、基本设计原则

有一些适用于部件级设计的基本原则，这些原则在使用面向对象软件工程方法时被广泛采用。**使用这些原则的好处在于，使得产生的设计在发生变更时能够适应变更并且减少副作用的传播。**

□ 开闭原则(OCP): 软件实体对扩展是开放的，对修改关闭。

即：在设计类时，应分析该类的稳定性，如果预期该类会根据业务发生较多变化，我们应该考虑采用“扩展”的方式应对变化，而尽可能避免采用“修改”原代码来实现变化。

一、部件级设计

2、基本设计原则

□ 开闭原则

□ 开闭原则举例

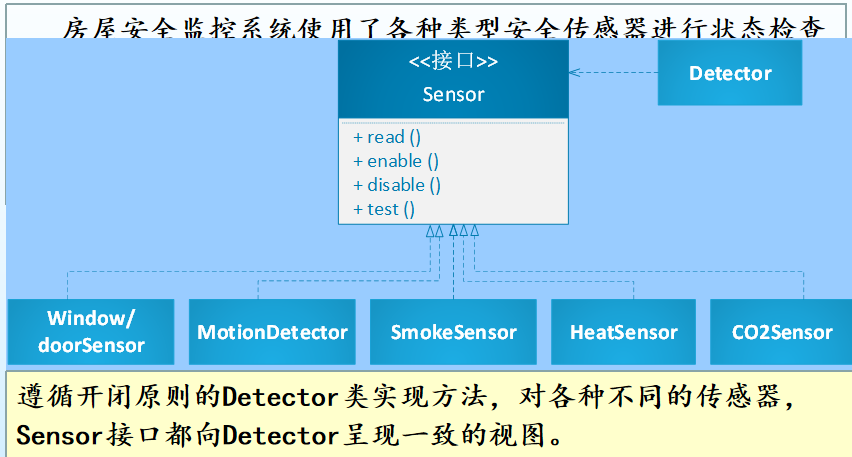
房屋安全监控系统使用了各种类型安全传感器进行状态检查的Detector类，随着时间的推移，安全传感器的类型和数量会不断增加，如果内部处理逻辑采用一系列if-then-else的结构来实现，其每个这样的结构都负责一个不同的传感器类型，那么这就违反了开闭原则。

一、部件级设计

2、基本设计原则

□ 开闭原则

□ 开闭原则举例



一、部件级设计

例：面向接口的编程

后花园对象实例

```
package cn.ccx.cretetype.simplefactory0.none;    创建葡萄类
public class Grape {
    public void plant() { System.out.println("葡萄种植..."); }
    public void harvest() { System.out.println("葡萄收获..."); }
```

```
package cn.ccx.cretetype.simplefactory0.none;    创建草莓类
public class Strawberry {
    public void plant() { System.out.println("草莓种植..."); }
    public void harvest() { System.out.println("草莓收获..."); }
}
```

创建客户端

一、部件级设计

例：面向接口的编程

后花园对象实例

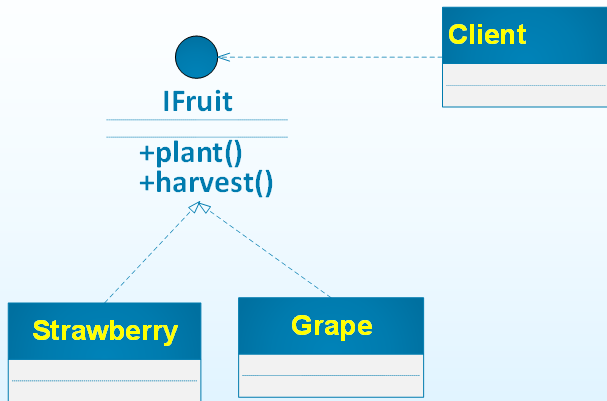
```
package cn.ccx.cretetype.simplefactory0.none;    创建葡萄类
public class Grape {
    public void plant() { Svsstem.out.println("葡萄种植..."); }
    public void harvest() { Svsstem.out.println("葡萄收获..."); }
```

```
package cn.ccx.cretetype.simplefactory0.none;
public class Client0 {
    public static void main(String[] args) {
        Grape grape=new Grape();
        Strawberry strawberry=new Strawberry();
        grape.plant();
        grape.harvest();
        strawberry.plant();
        strawberry.harvest(); }
}
```

创建客户端

一、部件级设计

例：面向接口的编程



一、部件级设计

例：面向接口的编程

```
package cn.ccx.cretetype.simplefactory1;  
public interface IFruit {  
    public void plant();  
    public void harvest(); }  
}
```

```
package cn.ccx.cretetype.simplefactory1;  
public class Strawberry implements IFruit {  
    public void plant() { System.out.println(“草莓种植...”); }  
    public void harvest() { System.out.println(“草莓收获...”); }  
}
```

创建客户端

一、部件级设计

例：面向接口的编程

```
package cn.ccx.cretetype.simplefactory1;  
public interface IFruit {  
    public void plant();  
    public void harvest(); }  
}
```

```
package cn.ccx.cretetype.simplefactory1;  
public class Client1 {  
    public static void main(String[] args) {  
        Ifruit[] fruits=(new Grape(),new Strawberry());  
        for(Ifruit fruit; fruits) {  
            fruit.plant();  
            fruit.harvest(); }  
    }  
}
```

一、部件级设计

2、基本设计原则

□ 开闭原则举例

THINKSYSTEM850....Info - dbo.User			
列名	数据类型	允许 Null 值	
userName	nchar(20)	☐	
password	nchar(20)	☐	
gender	nchar(10)	☒	
phone	nchar(20)	☐	
userType	int	☐	

在数据库中，使用字段userType来区分不同的会员类型，如0=普通会员，1=银卡会员，2=金卡会员。不同的会员类型享受不同的折扣率，普通会员=1.0，银卡会员=0.98，金卡会员=0.96

□ 开闭原则

@Entity

@Data

```
public class UserInfo{  
    @column  
    private String username;  
    @column  
    private String password;  
    @column  
    private int userType;  
    public double getDiscount() {  
        switch(userType) {  
            case 1: return 0.98;  
            case 2: return 0.96;  
            default: return 1.0  
        }  
    }  
}
```

一、部件级设计

2、基本设计原则

□ 开闭原则举例

THINKSYSTEM850....Info - dbo.User			
列名	数据类型	允许 Null 值	
userName	nchar(20)	☐	
password	nchar(20)	☐	
gender	nchar(10)	☒	
phone	nchar(20)	☐	
userType	int	☐	

在数据库中，使用字段userType来区分不同的会员类型，如0=普通会员，1=银卡会员，2=金卡会员。不同的会员类型享受不同的折扣。

问题：随着会员购买力上升，企业决定新增高阶的会员“铂金会员”，享受最高0.95折优惠。如何新增这类会员？

□ 开闭原则

@Entity

@Data

```
public class UserInfo {  
    @column  
    private String username;  
    @column  
    private String password;  
    @column  
    private int userType;  
    public double getDiscount() {  
        switch(userType) {  
            case 1: return 0.98;  
            case 2: return 0.96;  
            case 3: return 0.95;  
            default: return 0.9;  
        }  
    }  
}
```

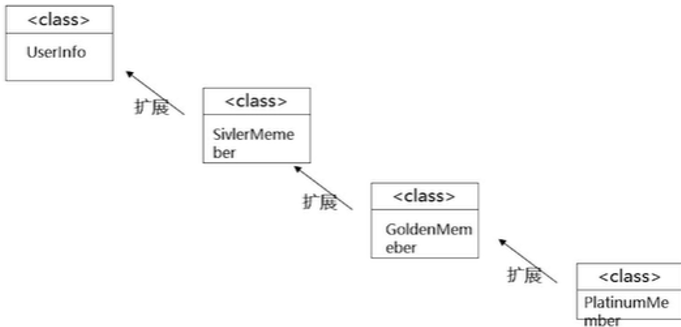
一、部件级设计

2、基本设计原则

□ 开闭原则

□ 开闭原则举例

我们可以遵循开闭原则进行类的设计



一、部件级设计

2、基本设计原则

□ 开闭原则

□ 开闭原则举例

我们可以遵循开闭原则进行类的设计



```
<class>
@Entity
@DiscriminatorColumn(userType= "userType" )
public class UserInfo{
    @column
    private String username;
    @column
    private String password;
    @column
    private int userType;
    public double getDiscount() {
        return 1.0
    }
}
```

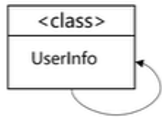
```
@Entity
@DiscriminatorValue(3)
public class PlatinumMember extends
GoldenMember {
    @Override
    public double getDiscount() {
        return 0.95
    }
}
```

一、部件级设计

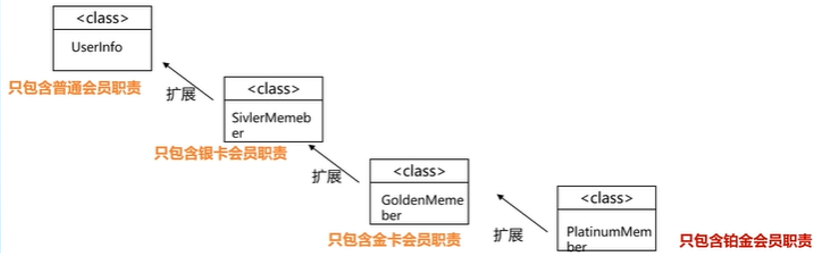
2、基本设计原则

□ 职责单一原则

□ 职责单一性原则(SRP): 就一个类而言, 应该仅有一个引起它变化的职责, 即一个类只做一件事。



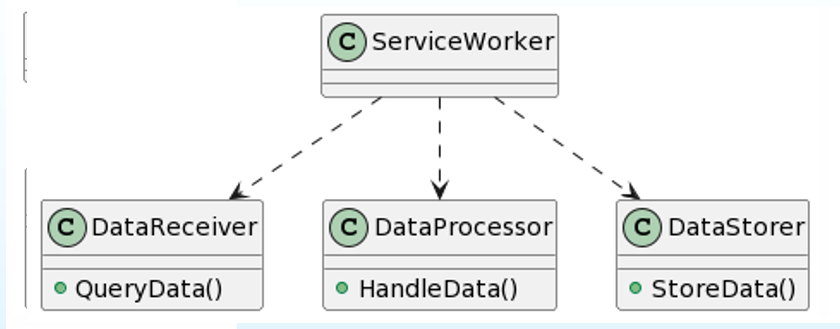
在上述开闭原则案例中, 普通会员、银卡会员、金卡会员之所以发生变化, 主要原因是他们的职责需要发生变化。具体地说, 就是getDiscount()方法, 不同的会员类型返回不同的值。



一、部件级设计

2、基本设计原则

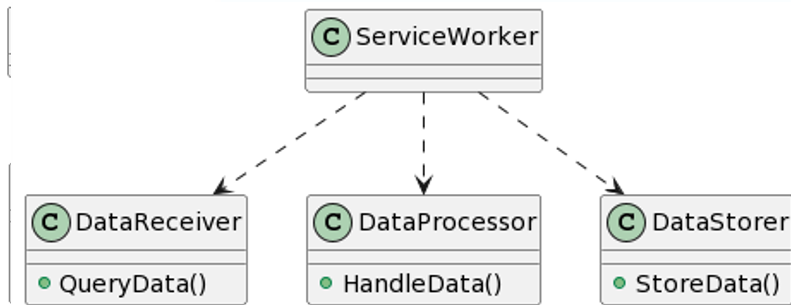
□ 职责单一原则



一、部件级设计

2、基本设计原则

□ 职责单一原则



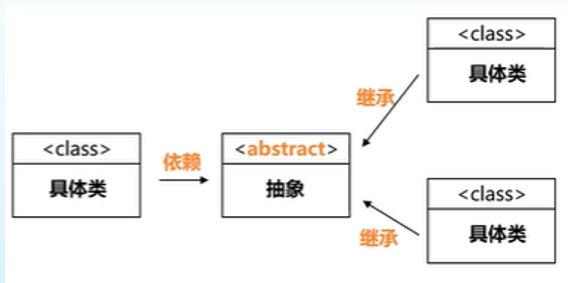
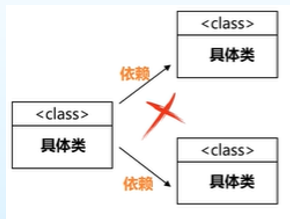
如果说QueryData中从服务器端请求的格式发生了变化，其实这个并不应该影响到HandleData和StoreData，因此他是可以剥离开来，专注于数据获取的操作

一、部件级设计

2、基本设计原则

□ 依赖倒置原则

□ 依赖倒转原则 (DIP) : 预期有较大变化可能的类间依赖, 应尽量依赖于抽象而不依赖于具体实现。 抽象可以比较容易地对设计进行扩展, 又不会导致大量的混乱。



一、部件级设计

2、基本设计原则

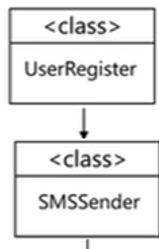
□ 依赖倒转原则 (DIP)

短信/彩信
前天 上午10:26

亲爱的会员，欢迎加入美丽芳丝会员俱乐部，您的手机号与会员卡号已绑定成功，即享积分换礼等多重会员专属权利。客服热线 [4006806600](tel:4006806600)/回TD退订【佳丽宝】

在CRM系统中，UserRegister类会在各类节日时，向会员发送短信，推荐各类优惠信息，信息通过SMSSender类向UserInfo类发送。

```
public class UserRegister {  
    private SMSSender smsSender;  
    public void recommendate(String info) {  
        smsSender.send(info)  
    }  
}
```

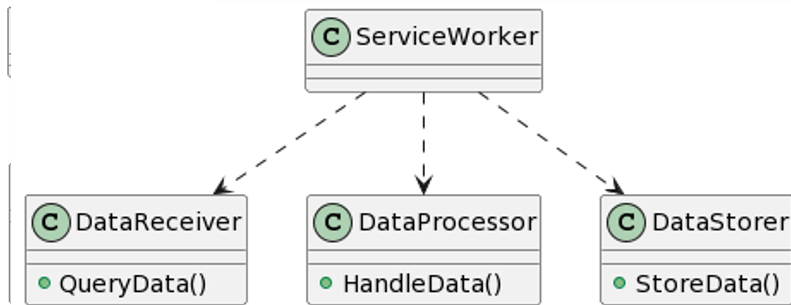


问题：后续上微信公众号(WechatSender)、微信服务号(WechatService)、小程序(MiniSender)发送推荐信息

一、部件级设计

2、基本设计原则

□ 职责单一原则



如果说QueryData中从服务器端请求的格式发生了变化，其实这个并不应该影响到HandleData和StoreData，因此他是可以剥离开来，专注于数据获取的操作

一、部件级设计

2、基本设计原则

□ 依赖倒转原则 (DIP)

要能够支持动态扩展，我们可以考虑移除UserRegister类中对SMSSender这个具体类的依赖。同时，抽象一个Sender类，其他具体的发送类，如WechatSender、WechatService、MiniSender，继承或实现这个抽象的Sender类。

```
public abstract class Sender {  
    //抽象方法  
    public void send(String info);  
}
```

```
public class UserRegister {  
    //引用抽象的Sender类  
    private Sender sender;  
    public void recommendate(String info) {  
        sender.send(info)  
    }  
}
```

一、部件级设计

2、基本设计原则

□ 依赖倒转原则 (DIP)

要能够支持动态扩展，我们可以考虑移除UserRegister类中对SMSSender这个具体类的依赖。同时，抽象一个Sender类，其他具体的发送类，如WechatSender、WechatService、MiniSender，继承或实现这个抽象的Sender类。

```
public abstract class Sender {  
    //抽象方法  
    public void send(String info) {  
    }  
}
```

```
public class WechatSender extends Sender {  
    @Override  
    public void send(String info) {  
        //通过微信发送推送信息  
    }  
}
```

```
public class UserRegister {  
    //引用抽象的Sender类  
    private Sender sender;  
    public void recommendate(String info) {  
        sender.send(info)  
    }  
}
```

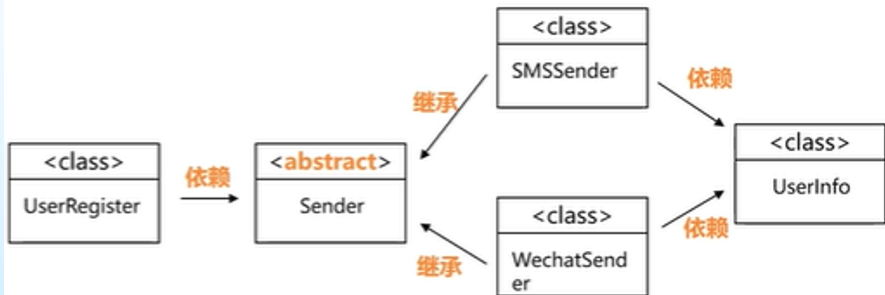
一、部件级设计

2、基本设计原则

□ 依赖倒转原则 (DIP)

要能够支持动态扩展，我们可以考虑移除UserRegister类中对SMSSender这个具体类的依赖。同时，抽象一个Sender类，其他具体的发送类，如WechatSender、WechatService、MiniSender，继承或实现这个抽象的Sender类。

```
public abstract class Sender {
```

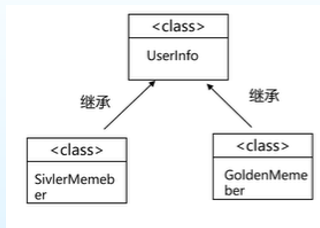


一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则

□ 组合/聚合复用原则：尽可能使用组合/聚合，而不是使用继承，来实现类的职责（行为方法的扩展）。



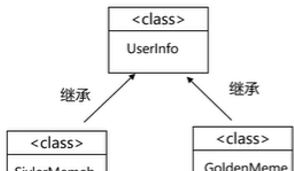
假设CRM系统中有三种会员，分别是普通会员、银卡会员、金卡会员，对应享受不同的折扣率，通过getDiscount()职责来实现。

一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则

□ 组合/聚合复用原则：尽可能使用组合/聚合，而不是使用继承，来实现类的职责（行为方法的扩展）。



假设CRM系统中有三种会员，分别是普通会员、银卡会员、金卡会员，对应享受不同的折扣率，通过getDiscount()职责来实现。

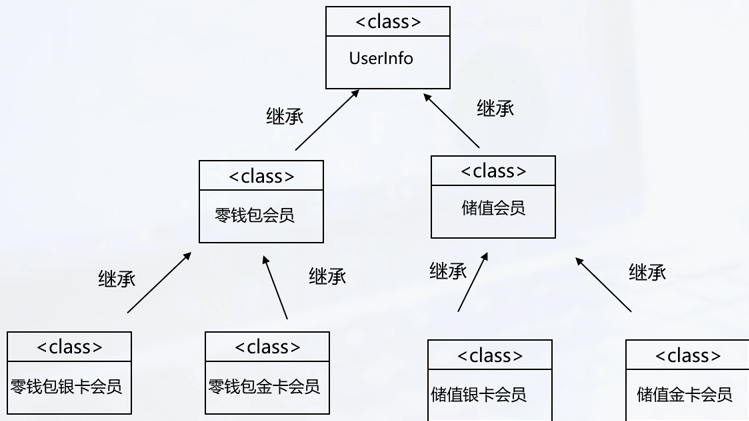
问题：业务需求发生了变化，需要区分零钱储值会员和电子储值会员。

零钱储值：付款的零头，可以自动存入零钱包，下次消费时，零头自动扣去零钱。

一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则

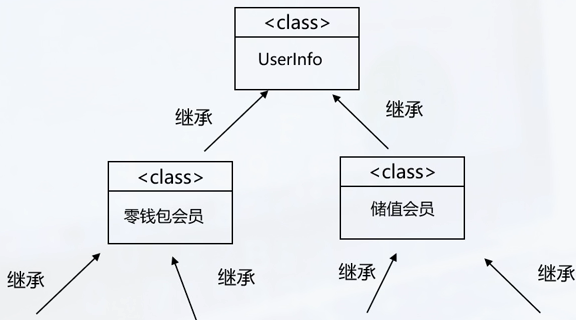


按照单一职责原则和开闭原则，我们不得不进行如上的设计

一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则



在实践中，继承的使用远远少于组合。除非有十分明确的分类，这些分类是通过“成员属性”来明确划分的，而不是通过“行为方法”来划分。

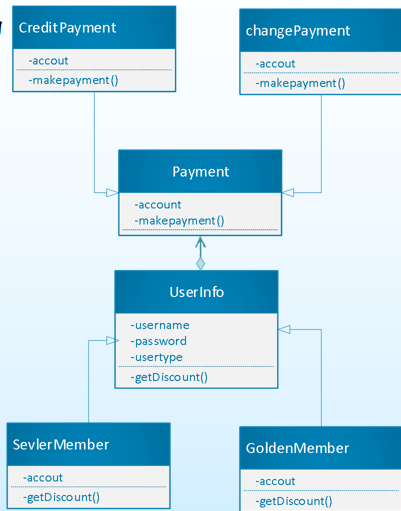
一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则

我们可以考虑用“组合/聚合”的方式来实现，具体做法：

- 1、定义一个能够表示付款的类Payment。
- 2、零钱包付款和储值付款都继承于这个类。
- 3、将Payment以对象耦合的方式聚合到User Info类
- 4、各类会员卡能够通过这个组合的Payment完成不同的付款。



一、部件级设计

2、基本设计原则

我们可以考虑用“
方式来实现，具体

1、定义一个能够表
Payment。

2、零钱包付款和储
承于这个类。

3、将Payment以对
聚合到User Info类

4、各类会员卡能够
合的Payment完成不同

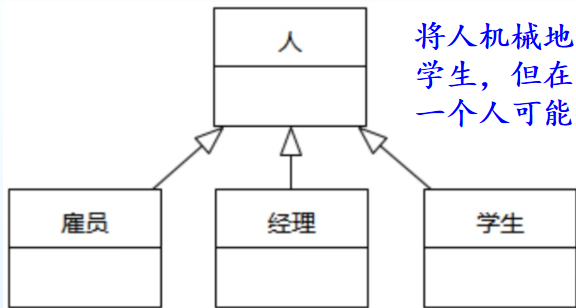
```
@Entity
@DiscriminatorColumn(userType= "userType" )
public class UserInfo{
    @column
    private String username;
    @column
    private String password;
    @column
    private int userType,
    //组合进来的储值/零钱付款职责
    private Payment payment;
    public void creditPayment(double amount) {
        payment.makePayment(amount);
    }

    public double getDiscount() {
        return 1.0;
    }
}
```

一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则

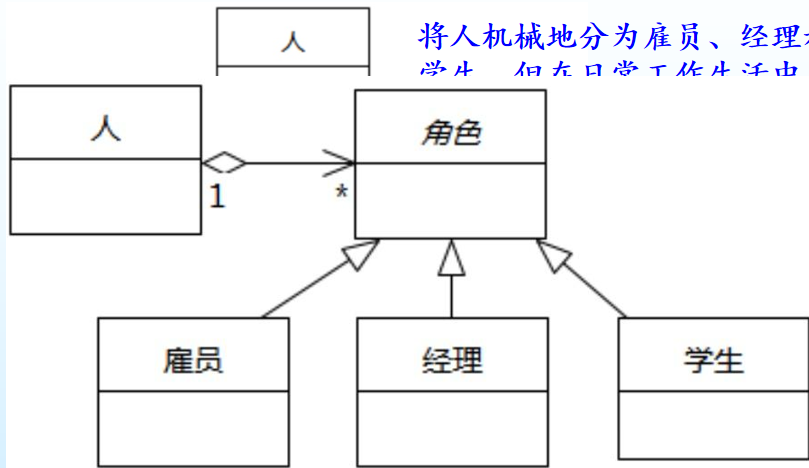


将人机械地分为雇员、经理和学生，但在日常工作生活中，一个人可能会分饰多种角色

一、部件级设计

2、基本设计原则

□ 组合/聚合复用原则



一、部件级设计

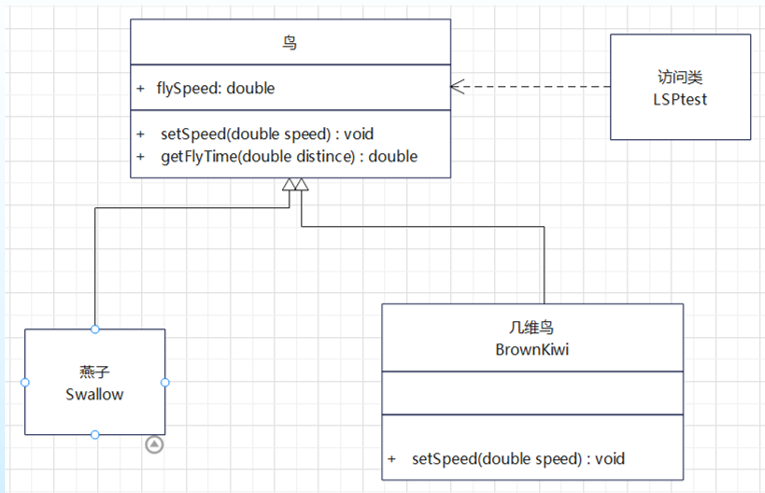
2、基本设计原则

里氏替换原则 (LSP) : 将从基类导出的类传递给构件时, 使用基类的构件应仍能够正确完成其功能。 子类可以扩展父类的功能, 但不能改变父类原有的功能。里氏代换原则是[开闭原则](#)的重要方式之一, 由于使用父类对象的地方都可以使用子类对象, 因此在程序中尽量使用父类类型来对对象进行定义, 而在运行时再确定其子类类型, 用子类对象来替换父类对象。

- 子类继承父类时, 除添加新的方法完成新增功能外, 尽量不要重写父类的方法。

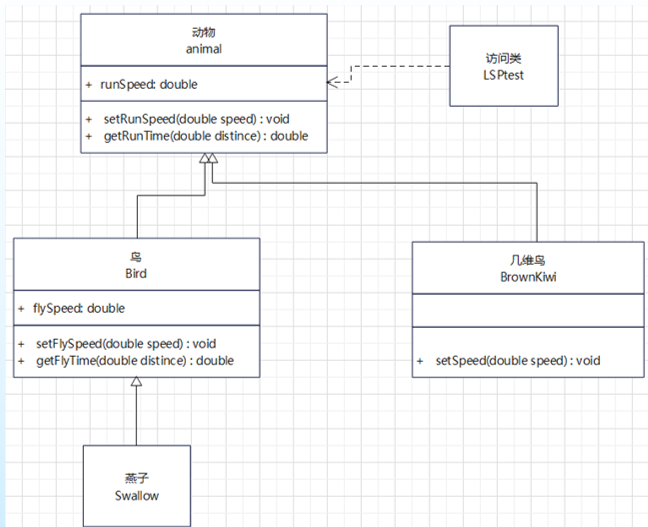
一、部件级设计

2、基本设计原则



一、部件级设计

2、基本设计原则



一、部件级设计

2、基本设计原则

- ❑ 接口隔离原则 (ISP) : 使多个接口相互隔离, 为客户端提供尽可能小的单一接口。 ISP原则建议设计者为每个主要的客户类型都设计一个特定的接口, 只有那些与特定接口类型相关的操作才应该出现在该客户的接口说明中。如果多个客户要求相同的操作, 则这些操作应该在每个特定的接口中说明。

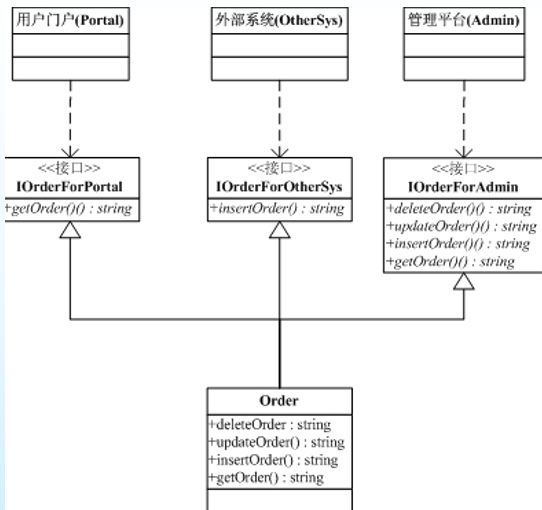
使用场合,提供调用者需要的方法,屏蔽不需要的的方法.满足接口隔离原则.
比如说电子商务的系统,有订单这个类,有三个地方会使用到,

- 一个是用户,只能有查询方法,
- 一个是外部系统,有添加订单的方法,
- 一个是管理后台,添加删除修改查询都要用到.

一、部件级设计

2、基本设计原则

根据接口隔离原则 (ISP), 一个类对另外一个类的依赖性应当是建立在最小的接口上. 也就是说, 对于用户, 它只能依赖有一个查询方法的接口



一、部件级设计

2、基本设计原则

- 迪米特法则：最少知道原则，一如果两个类不必直接通信，那么这两个类不应当发生直接的相互作用。如果其中一个类需要调用另一个类的某一个方法的话，可通过第三者发起这个调用。

如果一个系统满足迪米特法则，那么当其中一个软件实体发生变化时，就会尽量少的影响其他软件实体，扩展会相对容易，这是对软件实体之间通信的限制，迪米特法则要求限制软件实体之间通信的宽度和深度。迪米特法则可以降低系统的耦合度，使类与类之间保持松耦合状态。

二、人机界面设计

1、人机界面设计概述

人机界面也称为用户界面，是人与计算机之间传递和交互信息的媒介。



- 影响用户对软件的感觉
- 衡量软件可用性的标准之一
- 计算机科学与心理学、设计艺术学、认知科学和人机工程学的交叉研究领域。

二、人机界面设计

2、人机界面设计要考虑的因素

(1) 系统响应时间

系统响应时间指从用户完成某个控制动作(例如, 按回车键或单击鼠标), 到软件给出预期的响应(输出信息或做动作)之间的这段时间。

系统响应时间有两个重要属性, 分别是长度和易变性。

1) 长度: 时间过长, 用户就会感到紧张, 过短, 加快用户操作节奏, 可能会犯错误。

2) 易变性: 系统响应时间相对于平均响应时间的偏差。

即使系统响应时间较长, 响应时间易变性低也有助于用户建立起稳定的工作节奏。

二、人机界面设计

2、人机界面设计要考虑的因素

(2) 用户帮助设施

具体设计帮助设施时，必须解决下述的一系列问题。

1) 在用户与系统交互期间，是否在任何时候都能获得关于系统任何功能的帮助信息？

2) 用户怎样请求帮助？有3种选择：帮助菜单、特殊功能键和HELP命令。

3) 怎样组织帮助信息？有3种选择：平面结构，信息的层次结构和超文本结构。

4) 用户怎样返回到正常的交互方式中？有两种选择：屏幕上的返回按钮和功能键？

二、人机界面设计

2、人机界面设计要考虑的因素

(3) 出错信息处理

出错信息和警告信息，是出现问题时交互式系统给出的“坏消息”。一般说来，交互式系统给出的出错信息或警告信息，具有下述属性。

- 1) 用用户可以理解的术语描述问题；
- 2) 提供有助于从错误中恢复的建设性意见；
- 3) 指出错误可能导致哪些负面后果(如破坏数据文件)，以便用户检查是否出现了这些问题，并在确实出现问题时及时解决；
- 4) 伴随着听觉上或视觉上的提示；
- 5) 不能带有指责色彩，不能责怪用户。

二、人机界面设计

2、人机界面设计要考虑的因素

(4) 命令交互

许多高级用户仍然偏爱面向命令行的交互方式，在提供命令交互方式时，必须考虑下列设计问题：

1) 是否每个菜单选项都有对应的命令？

2) 采用何种命令形式？有3种选择：控制序列（例如，Ctrl+P），功能键和输入命令。

3) 学习和记忆命令的难度有多大？忘记了命令怎么办？

4) 用户是否可以定制或缩写命令？

在越来越多的应用软件中，人机界面设计者都提供了“命令宏机制”，在理想的情况下，所有应用软件都有一致的命令使用方法。

二、人机界面设计

3、人机界面的风格

□ 命令和询问方式的界面

- 通过用户命令和用户对象系统询问的响应来完成。
- 存在问题：用户容易出错，界面不友善，难以学习。



□ 简单的菜单式界面

- 用户只需确认菜单项而不需记忆系统命令。
- 下拉菜单需逐层进行不能一步到位。
- 比命令式界面便于用于使用，但要通过键盘来控制光标移动



二、人机界面设计

3、人机界面的风格

□ 窗口、图标、菜单、指示器四位一体界面

同时显式不同种类的信息，引入图标、下拉式菜单、按钮和滚动杆技术，大大减少键盘输入，提高交互效率。



□ 窗口、图标、菜单、指示器与超文本、多任务概念相结合的界面

用户可以同时执行多个任务。



二、人机界面设计

3、人机界面的风格



二、人机界面设计

3、人机界面的风格

应急指挥

主菜单 应急指挥 信息汇集 综合协调 事件管理 系统管理 测试页

当前操作人员信息

姓名	王伟	工号	001	登录名	admin
单位	1	部门	应急协调科	时间	08:10:50 09-21

报警信息

来源单位	某上报单位	文种	报告	编号	001
来电人	王某	电话	02022	时间	20:12:46
接电人	李某	电话	2883	启动状态	预警

事件基本信息

事件编号 20090921000011 事发日期 -- -- 事发时间 --:--:--

事件名称 群体性事件 事发地址 地图定位

事件类型 群体性事件 14A00 非法集会游行示威

人员数量 0 人员成分 影响范围

死亡数量 0 受伤人数 0 控制难度

事件级别 主责单位 选择

管辖区单位

事件处置记录

情况内容

处置情况

报告时间

信息窗

类型选择

名称

- 自然灾害
- 事故灾难
- 公共卫生事件
- 社会安全事件
- 其他突发事件

名称

- 核与辐射事故
- 能源供应中断事故
- 其他事故灾难
- 传染病事件
- 食品药品安全事件
- 群体性中毒、感染事件
- 动物疫情事件
- 病原微生物、菌毒株事件
- 群体性不明原因疾病
- 其它严重影响公共健康和...
- 其他公共卫生事件
- 群体性事件
- 刑事案件
- 金融突发事件
- 影响市场稳定的突发事件
- 民族和宗教事件
- 恐怖袭击事件

名称

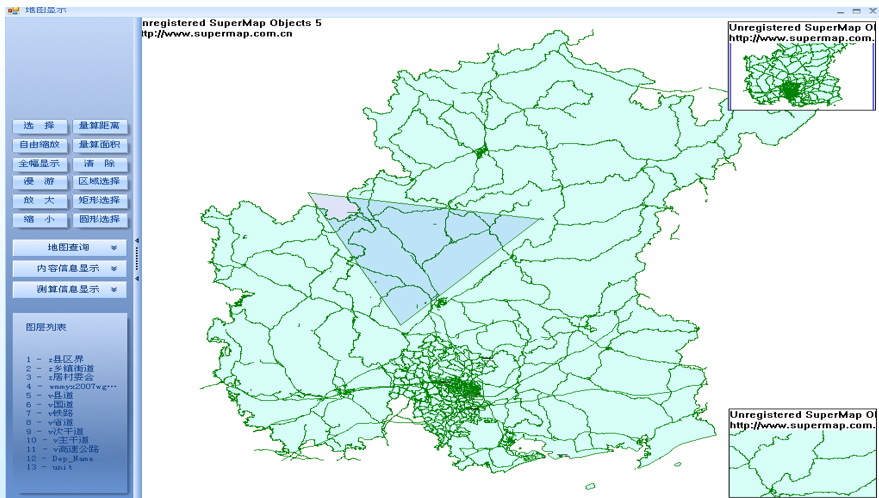
- 非法集会游行示威
- 集体上访请愿
- 冲击、围攻党政机关和要...
- 大规模打、砸、抢、烧泥...
- 群体性械斗、冲突事件
- 静坐事件
- 罢市、罢工
- 罢课
- 高校内聚集事件失控
- 高校校园网大范围串联、...
- 阻断交通事件
- 阻挠、妨碍国家重点建设...
- 暴乱事件
- 聚众闹事
- 其他群体事件

流程菜单

- 事件选取
- 事件新报
- 重报假报
- 事件定位
- 事件定级
- 事件定级
- 方案生成
- 力量调派
- 物资调派
- 事件上报
- 预案引入
- 方案执行
- 处置反馈
- 查听录音
- 事件保存
- 事件接管
- 事件续报
- 事件浏览
- 事件归档
- 事件变更
- 退出系统

二、人机界面设计

3、人机界面的风格

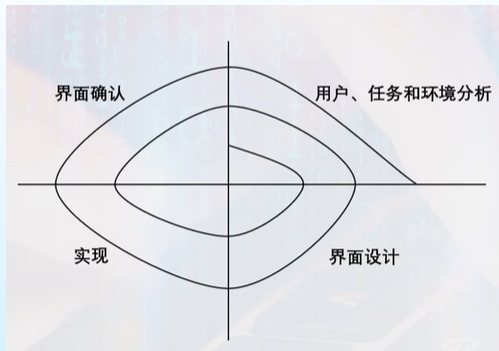


三、人机界面设计

4、人机界面设计过程

人机界面设计步骤

(1) 用户、任务和环境分析及建模：针对用户特点获取需求，分析任务、环境并建模。



(2) 界面设计：定义一组界面对象、动作以及它们的屏幕表示。

(3) 实现：根据设计方案，使用实现工具完成界面的构造。

(4) 界面确认：界面是否正确实现每个用户任务、是否容易使用和学习，用户是否接受。

三、人机界面设计

5、黄金原则

Theo Mandel在《The Elements of Interface Design》中提出的人机界面设计规则。

- ▶ **让用户拥有控制权：** 用户希望控制计算机，而不是被计算机控制。交互模式的定义不能强迫用户进入不必要的或不希望的动作方式；提供灵活的交互；允许用户交互可以被中断和撤销；当技能级别增长时可以使交互流水化并允许定制交互；使用户隔离内部技术细节。
- ▶ **减少用户的记忆负担：** 要求用户记住的东西越多，与系统交互时出错的可能越大。减少对短期记忆的要求；建立有意义的缺省；定义之直觉性的捷径；界面的视觉布局该基于真实世界的隐喻；以不断进展的方式揭示信息。
- ▶ **保持界面一致：** 用户应该以一致的方式展示和获取信息。允许用户将当前任务放在有意义的语境中；在应用系列内保持一致性；不要改变用户已经熟悉的用户交互模型。

谢谢观看

